TTS 가이드

베리어프리 키오스크에서 `tabIndex`와 `aria-label`을 사용하는 이유와 트러블슈팅 사례를 정리한 문서입니다.

> **Windows 내레이터와 로컬 TTS 모두 동일한 방식을 사용합니다.**
> TTS 엔진 종류에 관계없이, 렌더러에서는 tabIndex로 포커스 가능한 요소를 만들고 aria-label에 읽어줄 텍스트를 작성하는 동일한 방식으로 구현합니다. 엔진별 처리 차이는 내부에서 자동으로 처리되므로 개발자가 신경 쓸 부분은 없습니다.

왜 필요한가

시각 장애 사용자를 위한 TTS 음성 안내를 구현하기 위해, 두 가지 HTML 속성이 핵심적인 역할을 합니다.

> **핵심 원리:** tabIndex는 요소를 포커스 가능하게 만들고, aria-label은 포커스된 요소가 읽어줄 텍스트를 정의합니다. 이 두 속성이 함께 작동해야 TTS 음성 안내가 동작합니다.

속성	역할	없을 때 결과
`tabIndex`	요소에 포커스를 받을 수 있게 함	키보드 네비게이션으로 도달 불가, TTS 미작동
`aria-label`	TTS가 읽을 텍스트를 정의	포커스는 되지만 읽어줄 텍스트 없음, 무음 또는 태그명만 읽힘

일반 웹 접근성에서는 스크린 리더가 요소의 텍스트 콘텐츠를 자동으로 읽습니다. 하지만 키오스크 환경에서는 **방향키 기반 공간 네비게이션**을 사용하기 때문에, 포커스 관리를 직접 제어해야 합니다.

내부 동작 구조

포커스부터 음성 출력까지의 전체 흐름입니다. 아래 용어는 프로젝트 내부 컴포넌트입니다.

컴포넌트	설명
`Focusable`	자식 요소에 `tabIndex` 와 `aria-label` 을 동적으로 주입하는 래퍼 컴포넌트
`FocusableProvider`	앱 전체를 감싸며 키보드 이벤트 처리, aria-live 영역 관리, TTS 연동을 담당하는 Provider
`ScreenReader`	화면 밖(`left: -9999px`)에 배치되어 시각적으로 숨겨지지만 스크린 리더는 접근 가능한 styled-component
`FOCUS_ITEMS`	각 포커스 항목의 고유 키(`key`)와 음성 라벨(`label`)을 정의하는 상수 객체

```
키보드 입력 (방향키) → 공간 네비게이션 (setFocus) →
Focusable 컴포넌트가 aria-label 주입 → readCurrentFocus()로 텍스트 추출 →
aria-live 영역에 전달 → TTS 음성 출력
```

1단계: Focusable 컴포넌트의 aria-label 주입

`Focusable` 컴포넌트는 `cloneElement` 를 통해 자식 요소에 `tabIndex` 와 `aria-label` 을 동적으로 주입합니다.

Focusable.tsx - 핵심 로직 TypeScript

```typescript
return cloneElement(children, {
  ref,
  focused,
  tabIndex: -1,
  'aria-label': focusItem?.label,
  style: {
    ...children.props.style,
    outline: focused && focusVisible ? '4px solid #ffff00' : 'none',
    outlineOffset: '-3px',
  },
});
```

2단계: 포커스 시 aria-label 텍스트 추출

`FocusableProvider` 에서 현재 포커스된 요소의 `aria-label` 을 읽어 `aria-live` 영역에 전달합니다. 이 함수는 다시 읽기 키 입력, 음성 속도 변경, 볼륨 다이얼 조작 종료 등의 이벤트에서 호출됩니다.

FocusableProvider.tsx - 텍스트 추출 TypeScript

```typescript
const readCurrentFocus = () => {
  const focusedElement = document.activeElement as HTMLElement;
  if (focusedElement) {
    const ariaLabel = focusedElement.getAttribute('aria-label');
    if (ariaLabel) {
      announceText(ariaLabel);
    }
  }
};
```

3단계: aria-live 영역을 통한 음성 출력

숨겨진 `ScreenReader` 요소에 `aria-live="polite"`가 설정되어 있어, 텍스트가 변경되면 TTS 엔진이 자동으로 읽어줍니다.

FocusableProvider.tsx - aria-live 영역 TypeScript

```typescript
<ScreenReader
  ref={guideRef}
  role="status"
  aria-live="polite"
  aria-atomic="true"
/>

const announceText = (text: string) => {
  if (!guideRef.current) return;
  guideRef.current.textContent = '';
  requestAnimationFrame(() => {
    if (guideRef.current) {
      guideRef.current.textContent = text;
    }
  });
};
```

주의: `announceText`는 `textContent`를 빈 문자열로 초기화한 뒤 `requestAnimationFrame`으로 다시 설정합니다. 동일한 텍스트를 연속으로 읽어야 할 때 aria-live가 변경을 감지하도록 하기 위한 패턴입니다.

tabIndex의 역할

키오스크에서 `tabIndex`가 필요한 이유와, 일반 웹과의 차이점입니다.

일반 웹 vs 키오스크 환경

환경	네비게이션 방식	tabIndex 필요 여부
일반 웹	Tab 키로 순차 이동	`<button>`, `<a>` 등은 기본 포커스 가능
키오스크	방향키(상하)로 공간 이동	모든 요소에 명시적 `tabIndex` 필요

키오스크에서는 브라우저 기본 Tab 네비게이션을 사용하지 않고, 방향키 기반 공간 네비게이션을 사용합니다. `setFocus()`를 통해 프로그래밍적으로 포커스를 이동합니다.

tabIndex="-1"을 사용하는 이유

tabIndex="-1"은 "Tab 키로는 접근 불가, JavaScript로만 포커스 가능"을 의미합니다.
키오스크에서는 Tab 키를 사용하지 않고 방향키로 이동하기 때문에, `tabIndex={-1}`로 설정하여 Tab 순서에서 제외하되 `element.focus()`로 프로그래밍적 포커스를 가능하게 합니다.

TABINDEX={0} 사용 시

Tab 키 순서에 포함됨
방향키 네비게이션과 Tab 네비게이션이 충돌
포커스 순서가 예측 불가능해짐
사용자가 의도하지 않은 요소로 이동 가능

TABINDEX={-1} 사용 시

Tab 키 순서에서 제외
방향키 네비게이션만으로 포커스 제어
`setFocus()`로 정확한 이동 보장
공간 네비게이션 라이브러리와 완벽 호환

Focusable 컴포넌트가 자동 주입

개발자가 직접 `tabIndex`를 작성할 필요 없이, `Focusable` 컴포넌트가 `cloneElement`로 자동 주입합니다.

사용법 TypeScript

```
// Focusable로 감싸면 tabIndex={-1}이 자동 주입됨
<Focusable focusItem={FOCUS_ITEMS.MENU.FLOOR(false)}>
  <MenuButton>층별안내</MenuButton>
</Focusable>

// 위 코드는 내부적으로 아래와 동일하게 렌더링됨
```

```
<MenuButton
  tabIndex={-1}
  aria-label="층별안내: 확인 버튼을 누르시면 층별 안내 메뉴로 이동합니다."
>
  층별안내
</MenuButton>
```

ScreenReader 요소에서의 직접 사용

상세 페이지의 긴 설명은 `ScreenReader` 컴포넌트에 직접 `tabIndex={-1}` 을 설정하고 `ref.focus()` 로 포커스합니다.

이벤트 상세 페이지 TypeScript

```
<ScreenReader
  ref={guideRef}
  tabIndex={-1}
  aria-label={guideLabel}
/>

// 읽기 모드 진입 시 프로그래밍적으로 포커스
useEffect(() => {
  if (read) {
    setTimeout(() => {
      guideRef.current?.focus();
    }, 100);
  }
}, [read]);
```

tabIndex 트러블슈팅

tabIndex 관련 문제 상황과 해결 방법입니다.

문제 1: 포커스가 전혀 이동하지 않음

증상: 방향키를 눌러도 요소에 노란 아웃라인이 표시되지 않고, TTS도 동작하지 않음

잘못된 코드

```
<div>층별안내</div>
```

tabIndex도 없고 Focusable로 감싸지도

올바른 코드

```
<Focusable focusItem={...}>
  <div>층별안내</div>
</Focusable>
```

않음.
브라우저가 이 요소를 포커스 대상으로 인식하지 못함.

Focusable이 `tabIndex={-1}`을 자동 주입.

문제 2: Tab 키로 의도하지 않은 이동 발생

증상: 사용자가 Tab을 누르면 방향키 네비게이션 순서를 건너뛰고 엉뚱한 요소로 이동

잘못된 코드

```
<button tabIndex={0}>층별안내
</button>
```

`tabIndex={0}`은 Tab 순서에 포함됨.
방향키 기반 네비게이션과 충돌.

올바른 코드

```
<button tabIndex={-1}>층별안내
</button>
```

또는 `Focusable`로 감싸서 자동 처리.
Tab 순서에서 제외되어 충돌 없음.

문제 3: ScreenReader 요소에 포커스가 안 됨

증상: `guideRef.current?.focus()`를 호출해도 포커스가 이동하지 않음

잘못된 코드

```
<ScreenReader ref={guideRef}
  aria-label={label} />
```

`tabIndex` 속성이 없으면 `<div>`는 기본적으로 포커스를 받을 수 없음.

올바른 코드

```
<ScreenReader ref={guideRef}
  tabIndex={-1}
  aria-label={label} />
```

`tabIndex={-1}`이 있어야 `.focus()` 호출이 동작.

---

aria-label의 역할

`aria-label`이 키오스크 TTS 시스템에서 수행하는 역할과 대안 속성과의 비교입니다.

aria-label을 사용하는 이유

키오스크 요소의 시각적 텍스트와 음성 안내 텍스트는 다릅니다.
화면에는 "층별안내"로 표시되지만, 음성으로는 "층별안내: 확인 버튼을 누르시면 층별 안내 메뉴로 이동합니다."라고 안내해야 합니다. `aria-label`은 시각적 텍스트를 덮어쓰고 별도의 음성 텍스트를 제공합니다.

왜 textContent가 아닌 aria-label인가

방식	장점	단점
`textContent`	추가 속성 불필요	화면 표시 텍스트와 음성 텍스트를 분리할 수 없음
`aria-label`	시각적 텍스트와 독립적인 음성 텍스트 정의 가능	속성을 명시적으로 작성해야 함
`aria-describedby`	다른 요소의 텍스트를 참조	별도 요소 필요, 동적 텍스트에 부적합
`title`	브라우저 기본 지원	TTS 엔진이 일관성 없이 처리, 호버 툴팁 표시

동적 상태 반영

`aria-label`은 JavaScript 표현식을 사용하여 현재 상태를 실시간으로 반영할 수 있습니다.

focusItems.ts - 동적 라벨 생성 TypeScript

```
FLOOR: (selected: boolean = false) => ({
  key: 'menu-item-floor',
  label: `층별안내${selected ? ', 선택됨' : ''}: 확인 버튼을 누르시면 층별 안내 메뉴로 이
})

// selected=true  → "층별안내, 선택됨: 확인 버튼을 누르시면 층별 안내 메뉴로 이동합니다."
// selected=false → "층별안내: 확인 버튼을 누르시면 층별 안내 메뉴로 이동합니다."
```

* 위 코드의 라벨 텍스트는 예시이며, 실제 프로젝트에서는 기관명 등 구체적인 내용이 포함됩니다.

복잡한 설명을 위한 generateLabel 유틸리티

단순 버튼이 아닌 상세 페이지에서는 `generateLabel` 유틸리티를 사용해 긴 설명 텍스트를 구조적으로 생성합니다.

generateLabel.ts - 이벤트 상세 예시 TypeScript

```
export const generateEventDesc = (event: BarrierFree.IEvent) => {
  const parts = [
    `${event.ko} 행사 안내입니다.`,
    `${event.time}에 ${event.position}에서 진행됩니다.`,
    event.price && `티켓은 ${event.price}이며`,
    event.age && `${event.age}입니다.`,
    event.needTime && `총 공연시간은 ${event.needTime}입니다.`,
    event.host && `이 공연은 ${event.host}에서 주최합니다.`,
    // ... 문의처 안내 등
    '확인 또는 취소 버튼을 눌러 이전 메뉴로 가실 수 있습니다.',
  ].filter(Boolean);

  return parts.join(' ');
};
```

aria-label 트러블슈팅

aria-label 관련 문제 상황과 해결 방법입니다.

문제 1: 포커스는 되지만 아무 소리도 안 남

증상: 노란색 아웃라인은 표시되지만 TTS 음성이 재생되지 않음

잘못된 코드

```
<Focusable focusItem={{
  key: 'menu-floor'
}}>
```

focusItem에 label 속성이 없음. aria-label이 undefined로 설정됨.

올바른 코드

```
<Focusable focusItem={{
  key: 'menu-floor',
  label: '층별안내: ...'
}}>
```

label 속성이 있어 aria-label에 텍스트 주입됨.

문제 2: 같은 텍스트를 연속으로 읽어주지 않음

증상: H 키를 눌러 다시 읽기를 해도 같은 내용이면 TTS가 반응하지 않음

`aria-live` 영역은 텍스트가 **변경**될 때만 읽어줍니다. 동일한 텍스트를 다시 설정하면 변경으로 감지되지 않습니다.

잘못된 방식

```
guideRef.current.textContent =
text;
```

이전과 같은 텍스트이면 aria-live가 변경을 감지하지 못해 음성이 재생되지 않음.

올바른 방식 (프로젝트 구현)

```
guideRef.current.textContent =
'';
requestAnimationFrame(() => {
  guideRef.current.textContent =
text;
});
```

빈 문자열로 초기화 후 다음 프레임에서 재설정하여 변경 감지.

문제 3: 복잡한 라벨에서 일부 정보가 누락됨

증상: generateLabel에서 조건부 필드가 undefined로 포함되어 "undefined"가 읽힘

잘못된 코드

```
const parts = [
  `${event.ko} 행사`,
  `티켓: ${event.price}`,
];
```

event.price가 없으면 "티켓: undefined"가 됨.

올바른 코드

```
const parts = [
  `${event.ko} 행사`,
  event.price && `티켓:
${event.price}`,
].filter(Boolean);
```

조건부 포함 + filter(Boolean)으로 falsy 값 제거.

문제 4: 상태 변경 후 라벨이 업데이트되지 않음

증상: 메뉴를 선택해도 "선택됨"이 aria-label에 반영되지 않음

잘못된 코드

```
<Focusable focusItem=
{FOCUS_ITEMS.MENU.FLOOR(false)}>
```

올바른 코드

```
<Focusable focusItem={
  FOCUS_ITEMS.MENU.FLOOR(currentMenu
```

selected 상태를 항상 false로 고정. 선택 상태가 반영되지 않음.

```
=== 'floor')
}>
```

실제 상태값을 전달하여 라벨이 동적으로 변경됨.

접근 방식 비교

TTS 구현 시 고려할 수 있는 여러 접근 방식과 현재 프로젝트에서 `tabIndex` + `aria-label` 을 선택한 이유입니다.

접근 방식 A: textContent 직접 읽기

textContent 방식 TypeScript

```typescript
// 포커스된 요소의 텍스트를 그대로 TTS에 전달
const text = focusedElement.textContent;
speak(text);
```

문제:

- 화면에 "층별안내"만 표시되므로 TTS도 "층별안내"만 읽음
- 어디로 이동하는지, 어떤 동작을 하는지 추가 안내 불가
- 아이콘만 있는 버튼(`<CloseIcon />`)은 읽을 텍스트 자체가 없음
- 자식 요소가 많으면 불필요한 텍스트까지 모두 합쳐서 읽힘

접근 방식 B: data-* 커스텀 속성

data 속성 방식 TypeScript

```typescript
<button data-tts="층별안내: 확인 시 이동합니다.">
  층별안내
</button>
```

문제:

- TTS 엔진을 자체 구현하면 `data-*` 속성을 읽는 것은 가능하지만, Windows 내레이터는 자체 스크린 리더를 활용하여 포커스된 요소의 `aria-label` 을 읽기 때문에 `data-*` 속성을 인식하지 못함
- 웹 접근성 표준(WCAG)에 부합하지 않음

접근 방식 C: aria-label (현재 채택)

aria-label 방식 TypeScript

```typescript
<button
  tabIndex={-1}
  aria-label="층별안내: 확인 버튼을 누르시면 층별 안내 메뉴로 이동합니다."
>
  층별안내
</button>
```

장점:

- W3C WAI-ARIA 표준 속성으로 스크린 리더 호환성 보장
- Windows 내레이터가 자동으로 인식하여 읽어줌
- 시각적 텍스트와 독립적으로 음성 텍스트를 정의 가능
- 로컬 TTS 엔진에서도 `getAttribute('aria-label')` 로 일관되게 추출
- 동적 상태를 JavaScript 표현식으로 즉시 반영 가능

종합 비교 표

기준	textContent	data-*	aria-label
스크린 리더 호환	부분적	불가	**완전 지원**
음성/시각 텍스트 분리	불가	가능	**가능**
WCAG 준수	부분적	미준수	**준수**
동적 상태 반영	가능	가능	**가능**
아이콘 전용 버튼	불가	가능	**가능**

용어 정의

이 문서에서 사용하는 주요 용어의 정의입니다.

용어	정의
`aria-label`	WAI-ARIA 속성. 요소의 접근 가능한 이름(Accessible Name)을 정의하며, 시각적 텍스트와 독립적인 라벨을 제공한다.
`aria-live`	WAI-ARIA 속성. 동적으로 변경되는 콘텐츠 영역을 스크린 리더에 알려, 변경 시 자동으로 읽어준다. `polite` (현재 읽기 후)와 `assertive` (즉시) 값이 있다.
`aria-hidden`	WAI-ARIA 속성. `"true"` 로 설정하면 스크린 리더가 해당 요소를 무시한다. 장식 요소에 사용한다.
`tabIndex`	HTML 속성. 요소의 포커스 가능 여부와 Tab 순서를 결정한다. `-1` 은 JavaScript로만 포커스 가능, `0` 은 Tab 순서에 포함.
`role`	WAI-ARIA 속성. 요소의 역할을 명시한다. 예: `dialog` , `alert` , `status` , `tab` , `img` .
TTS (Text-to-Speech)	텍스트를 음성으로 변환하는 기술. Windows 내레이터, ONNX 기반 로컬 엔진 등이 있다.
스크린 리더	화면의 텍스트와 UI 정보를 음성으로 읽어주는 소프트웨어. Windows 내레이터, NVDA, JAWS, VoiceOver 등이 있다.
공간 네비게이션	방향키(상하좌우)를 사용하여 화면의 요소 간 포커스를 이동하는 방식. 키오스크 환경에서 Tab 네비게이션 대신 사용한다.
접근 가능한 이름 (Accessible Name)	스크린 리더가 요소를 식별할 때 사용하는 텍스트. `aria-label` , `aria-labelledby` , 또는 요소의 텍스트 콘텐츠에서 결정된다.
WCAG	Web Content Accessibility Guidelines. W3C에서 발행한 웹 접근성 국제 표준 지침이다.

용어	정의
WAI-ARIA	Web Accessibility Initiative - Accessible Rich Internet Applications. W3C에서 정의한 접근성 강화 명세이다.